Practica de laboratorio 7:
HERENCIA - PRIMERA PARTE

Hernández Aviña Karen Itzel
Hernández González Mauricio

CAPTURAS DE SALIDA

```
spide@Muzan MINGW64 ~/MPOO/mpoo-2027-1/entregas/hernandez_mauricio/p06 (p04-hernandez)
$ java MainAnimales
Lomito, edad 0 dice ...
Firulais, edad 3 | criollo dice Guau
Max, edad 5 | labrador| asiste a Marta dice Guau
Michi, edad 2 | de interior  dice Miau
Max es labrador, edad 5
```

```
spide@Muzan MINGW64 ~/MPOO/mpoo-2027-1/entregas/hernandez_mauricio/p06 (p04-hernandez)
$ java MainPersonajes.java
Aldeano | vida  100 ataca con 10
Brakus | vida  80| fuerza 15 ataca con 25
Lunara | vida  0| fuerza 12|fe 40 ataca con 42
Zafiro | vida  60| mana 50 ataca con 25
```

CAPTURA DE SALIDA CON EL PRINTLN

```
kihav@lil_kraken17 MINGW64 /c/mpook/mpoo-2027-1/
en)
$ java MainPersonajes
Personaje listo
Personaje listo
Caballero listo
Personaje listo
Caballero listo
Paladin listo
Personaje listo
Aldeano | vida  100 ataca con 10
Brakus | vida  80| fuerza 15 ataca con 25
Lunara | vida  0| fuerza 12|fe 40 ataca con 42
Zafiro | vida  60| mana 50 ataca con 25
Lunara tiene fuerza 12 y fe 40
```

¿En qué orden salen las líneas al crear a Lunara, y porque?
Los textos de la clase “Paladín” son los primeros en salir antes que los textos marcados en la clase main debido al concepto de la herencia.
Donde el sistema primero busca correr los mensajes de la clase padre y después correr los mensajes de la clase hija que es la clase main que toma el constructor de la clase Paladín.
¿Por qué al crear a Brakus salen dos líneas y al crear a Lunara tres?
Esto se debe a la jerarquía que siguen ambos constructores.
Empezando por Brakus, este constructor solamente sigue una jerarquía de dos clases, pasando primero por la clase “Personaje” que crea la primera línea de texto y luego pasando por la segunda clase “Caballero” que nos da la segunda línea de texto.

Donde después tenemos a Lunara, que sigue el mismo procedimiento que Brakus solo diferenciándose que Lunara tiene una jerarquía que pasa por 3 clases.

Practica de laboratorio 7:
HERENCIA - PRIMERA PARTE

Pasando primero por la clase “Personaje” que crea la primera línea de texto, luego pasando por la clase “Caballero” que crea la segunda línea y por último pasando por la clase “Paladin” que es la que crea la última línea de texto.

En esta práctica fue fundamental entender la diferencia que existe entre una clase padre y una clase hija, como una clase hija con @Override puede mapear los cambios que queremos sobreescribir de un método heredado, es importante saber cuando usar private,public y protected,ya que cada modificador nos permite usar ese método o variables en otras clases,o por el contrario no dejar que se use esa información en otras clases.
Lo más difícil fue identificar exactamente qué es lo que estamos heredando de la clase padre para así sobreescribirlo en la clase hija, debemos de cuidar el nombre de los getter de cada clase porque si se heredan a una clase y ponemos un nombre diferente no podremos hacer el uso de esos métodos. No poner super en donde se necesita complica las cosas o ponerlo de manera incorrecta, tanto en el constructor de una clase que hereda, como al momento de hacer @Override ya que si ni lo colocamos no compila.